☆ Good for both intense short sessions and long term retention

☆ Easy to implement using open-source python libraries

Core Architecture:-

BKT (Byesian) → use for real time mastery tracking
- Tracks prob user knows the topic after each iteration

FSRS → Long term + intra session scheduling
- data driven replacement for SM-2
- Handles Both Cram sessions as well as multi-day gap sessions
- Uses difficulty per topic | timestamps | Stability

UCB (Upper Confidence Bound) → Topic selector during the session
- explore | exploit algorithm that chooses next topic
- Balances weak topics and less seen topics
- Cleaner than epsilon decay

Loop for a Single Session:-

☆ Log In → Load saved user BKT states + FSRS History

- Determine active topics (Based off FSRS or BKT with low mastery)

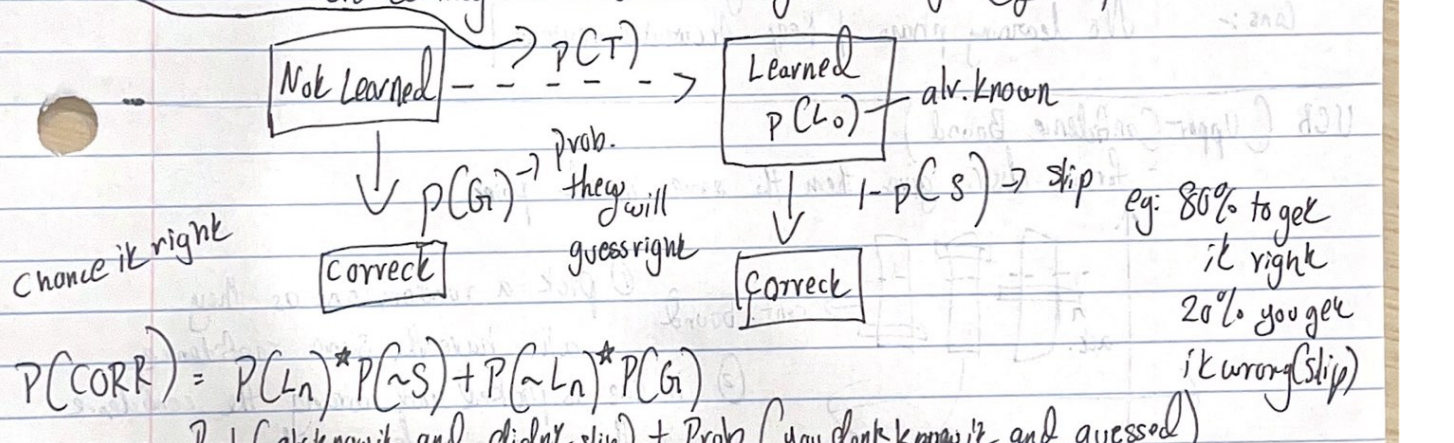
- While Session is active:-
  - Filter out mastered topics
  - Use UCB on top of BKT to pick next best topic (Scores)
  - Serve a question → User Answers → update BKT
  - Record Review with timestamp for FSRS
- End of Session → Update FSRS parameters and save Everything

Key Goal of BKT :- measure how well a student knows something at a specific time | Based on their past history of performance with that skill (Question)

Baysian (Brain of Program) (measure a specific skill)

- Each question is a single skill and each are scored (eg :- Student get 0 or 1)
  Student can learn each item (latent)

Assumption :- * Each item involves a single skill
* Each skill will have four parameters
  Based on here parameters and student's right or wrong answers compute :-
  $P(L_n)$ | Latent knowledge :- prob. the student alr knows it
  $P(CORR)$ | they will get it correct
* Each skill is either learned or unlearned
* During Problem-Solving during each question they have the ability to learn that topic
* Once they know a topic ideally wont forget it (No guarantee)
  - Chance they still get it wrong (slip)
  - chance they dont know it and get it right (guess)



$$P(CORR) = P(L_n) * P(\sim S) + P(\sim L_n) * P(G)$$

Prob (alr know it and didnt slip) + Prob (you dont know it and guessed)

(Bayes Theorem used) Whenever a question is asked, The prob. the students know the question is updated

$$P(L_{n-1} \mid Correct_n) = \frac{P(L_{n-1})(1-P(S))}{P(L_{n-1})(1-P(S)) + (1-P(L_{n-1}))(P(G))}$$

$$P(L_{n-1} \mid Incorrect_n) = \frac{P(L_{n-1}) * P(S)}{P(L_{n-1})(P(S)) + (1-P(L_{n-1}))((1-P(G)))}$$

$$P(L_n \mid Action_n) = P(L_{n-1} \mid Action_n) + ((1 - P(L_{n-1} \mid Action_n)) * P(T))$$

FSRS : based on (Difficulty, Stability, Retrievability) DSR model

- Free Spaced Repitition Scheduler

After first review :- Stability (grade :- again, hard, good, easy)
estimated during optimization

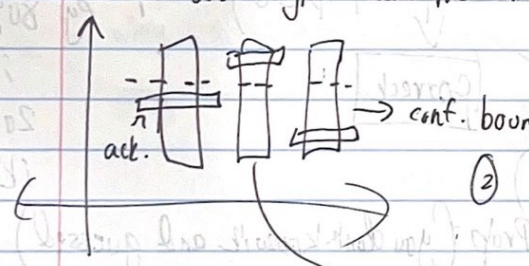
- intial difficulty also based on grade
- Next review based on Stability and desired retrievability (you pick retention)

Not first :- Calc Predicted Retrievability at time of review depending on Δt and Stability
Calc Difficulty based on prev and most recent grade
Calc new Stability using :- Difficulty, Stability, Retrievability
Stability goes ↓ then "Again"

Cons :- No learning phase | Req. Accurate reviews |

UCB (Upper-Confidence Bound)

- for all distr. give them the same initial point



① pick a random one as they all have the same confidence

② if one is picked and wrong the confidence in that topic is decreased
- The bound also gets smaller

③ over each iteration the value gets closer to the actual answer

④ once the expected value is reached or closer the bound decreases ~~in~~ even more